

EEE 282



Course Title: Numerical Techniques

Term Project Report

Section: 1.

Group No: 04

Name of the Project:

Coding Assignment.

All Group members:

S	Name	ID	Signature
1.	Souvik Barman Ratul	24121205	Ratul
2.	Alif Tamjid	24121308	Alif
3.	Sabrina Maliha	23221082	Sabrina
4.	Monazzi Siddique	24121019	Monazzi

Objective : The aim of this experiment is to correct the sampling error following a suitable interpolation method and to find the missing voltage value for accurate signal reconstruction.

Question 1

A student is analyzing a voice signal containing frequency components of up to 4 kHz. At the time of sampling the signal, s/he chose the Sample Time, $T_s = 0.25 \text{ ms}$ and the obtained the following data table:

Table - 1

$v \text{ (volts)}$	0	2.696	3.939	4.511	4.775
$t(\text{ms})$	0	0.25	0.5	0.75	1

But according to Nyquist's Sampling Theorem, s/he must sample the signal with a sampling rate of at least twice the maximum frequency component of the signal. So, the sampling frequency must have to be at least 8 kHz and thus the Sample Time, T_s should be 0.125 ms, for perfect reconstruction (i.e. the signal values should exist at time $t = 0 \text{ ms}$, 0.125 ms, 0.25 ms, 0.375 ms, 0.5 ms ... and so on).

- Can you help the student to correct the error? If so, which numerical technique are you going to use? Explain in brief.
- Derive the equation of voltage as a function of time considering a suitable Numerical approach (use Table – 1). Then, complete the following data table (Table – 2) with the unknown voltage values.

Table - 2

$v \text{ (volts)}$	0	?	2.696	?	3.939	?	4.511	?	4.775
$t(\text{ms})$	0	0.125	0.25	0.375	0.5	0.625	0.75	0.875	1

(i)

Answer:

Code :

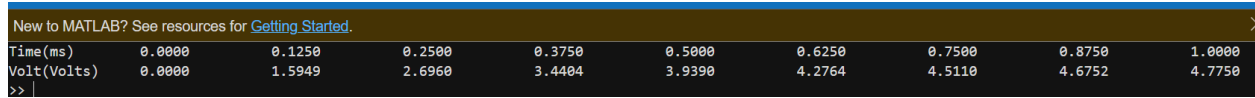
Main Script:

```
clear all;
close all;
clc;
v = [0 2.696 3.939 4.511 4.775];% sampling array
t = [0 0.25 0.5 0.75 1];% sampling array
Time_s = 0:0.125:1;% sampling array with step size 0.125.
Num = length (Time_s);%Number of elements in Time_s.
Voltage = volt_f_t (v,t,Time_s,Num);%Function calling for interpolation.
%To show result.
fprintf("Time(ms)\t");%Showing the result.
fprintf("%.4f\t\t",Time_s);
fprintf("\n");
fprintf("Volt(Volts)\t");
fprintf("%.4f\t\t",Voltage);
fprintf("\n");
```

Function:

```
function vol = volt_f_t(V,T,T_s,N)
n = length (T);
vol = zeros (1,N);% Allocating array to store values.
for k=1:N %For every value of the interpolated values along x axis.
P = ones(1,n);% Allocating array to store values.
for i = 1:n
for j = 1:n
if j~=i
P(i) = P(i) * (T_s(k)-T(j))/ (T(i) - T(j));
end
end
P(i) = P(i) * V(i);
end
vol(k) = sum(P);%Lagrange interpolation formula applied.
end
```

Output :



Time(ms)	0.0000	0.1250	0.2500	0.3750	0.5000	0.6250	0.7500	0.8750	1.0000
Volt(Volts)	0.0000	1.5949	2.6960	3.4404	3.9390	4.2764	4.5110	4.6752	4.7750

Figure 1(a): Displaying the values of voltage and time with step size of 0.125ms.

Time (ms)	0.0000	0.1250	0.2500	0.3750	0.5000	0.6250	0.7500	0.8750	1.0000
Volt (Volts)	0.0000	1.5949	2.6960	3.4404	3.9390	4.2764	4.5110	4.6752	4.7750

(b)

Code :

Main Script:

```
clear all;
close all;
clc;
v = [0 2.696 3.939 4.511 4.775];% sampling array
t = [0 0.25 0.5 0.75 1];% sampling array
Time_s = 0:0.125:1;% sampling array with step size 0.125.
Num = length (Time_s);%Number of elements in Time_s.
Voltage = volt_f_t (v,t,Time_s,Num);% Calling function for interpolation.
%To show result.
fprintf("Time(ms)\t");
fprintf("%.4f\t ",Time_s);
fprintf("\n");
fprintf("Volt (Volts)\t");
fprintf("%.4f \t",Voltage);
fprintf("\n");
for L = 2:2:8 %To find the missing value.
Volt = Voltage(L);% Value of voltage at every L.
fprintf("For time =%.3f ms, Volt=%.3f Volt.\n",Time_s(L),Volt);
end
figure(1)
plot(Time_s,Voltage,'r.','MarkerSize',20);%Plotting the graph.
title("Volts vs Time");% To give title over the graph.
xlabel("Time (ms)");% Labeling x axis.
ylabel("Volt (Volts)");%Labeling y axis.
hold on;% To keep the previous plot intact.
grid on;%To get grid lines.
Time_s2 = 0:0.0001:1;%Sampling array with step size of 0.0001.
N2 = length(Time_s2);
```

```
Voltage2 = volt_f_t(v,t,Time_s2,N2);%Calling function for interpolation.
plot(Time_s2,Voltage2,'b. ');% Plotting in the graph.
hold on;
```

Function:

```
function vol = volt_f_t(V,T,T_s,N)
n = length (T);
vol = zeros (1,N);% Allocating array to store values.
for k=1:N %For every value of the interpolated values along x axis.
P = ones(1,n);
for i = 1:n
for j = 1:n
if j~=i
P(i) = P(i) * (T_s(k)-T(j))/ (T(i) - T(j));% Allocating array to store values.
end
end
P(i) = P(i) * V(i);
end
vol(k) = sum(P);%Lagrange interpolation formula applied.
end
```

Output :

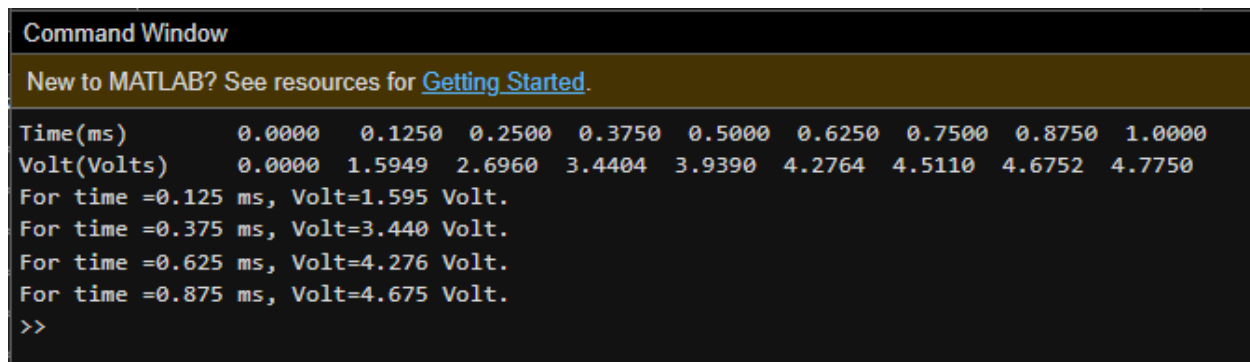


Figure 1(b)(i): Output showing the missing values as well as the construction of the table 2.

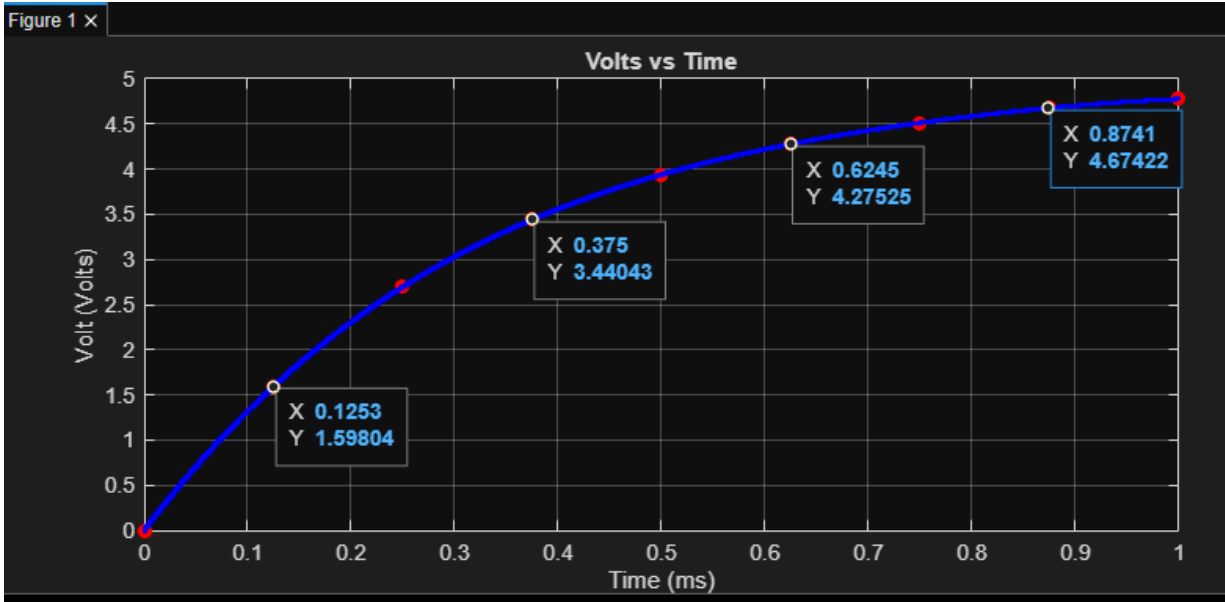


Figure 1 (b)(ii): Plot showing the volt (volts) vs Time(ms) graph.

Discussion:

The question says that the original sampling interval of 0.25ms was not correct fully according to Nyquist's Sampling Theorem for a signal containing frequency components of 4kHz. A numerical interpolation technique was used to correct the sampling error and estimate the missing signal value at a sampling interval of 0.125ms. Lagrange polynomial interpolation was chosen, as it is quite suitable to construct intermediate values using only certain finite data points without any knowledge of the signals's analytical form.

Table 1 was used to get an interpolation based volt-time equation and to compute the unknown values. Table 2 was completed using the same interpolation function. The plotted data confirms that the data derived in Table 2 constructed a continuous and smooth signal, which is supposed to be done for a signal with the characteristics of a properly sampled low frequency signal. Overall, the process successfully corrected the data and demonstrated the importance of Nyquist's sampling theorem and the effectiveness of interpolation technique in signal reconstruction.

Objective : The main objective of this experiment is to determine the RMS value of a voltage signal by using a suitable integration technique and to evaluate the accuracy of the result.

Question 2

The next task of the student is to calculate the RMS value of the signal. For any signal $v(t)$, RMS value is defined as:

$$V_{RMS} = \sqrt{\frac{1}{t_2 - t_1} \int_{t_1}^{t_2} v^2(t) dt}$$

- i. Determine the RMS value of the voltage signal from the completed data table (Table – 2) obtained from Question 1. Which numerical approach would be more appropriate for this scenario to get better accuracy? If the true value is 3.6908, determine the error of your Numerical approach.
- ii. Again, determine the RMS value of the voltage signal from the equation of the voltage obtained from Question 1(ii). Which numerical approach would you choose now to ensure better accuracy? Considering the true value mentioned previously, determine the error in this case.

Answer:

(i)

As we are dealing with sampled voltage values at specific time intervals, the Trapezoid Rule may be more appropriate here.

Code:

```
clear all;
close all;
clc;
t = [0, 0.125, 0.25, 0.375, 0.5, 0.6250, 0.7500, 0.8750, 1];      %% time (ms)
v = [0, 1.5949, 2.6960, 3.4404, 3.9390, 4.2764, 4.5110, 4.6752, 4.7750];  %% Volt(Volts)

xi = t(1);    %% lower limit
xf = t(end);  %% upper limit
h = 0.125;    %% step size

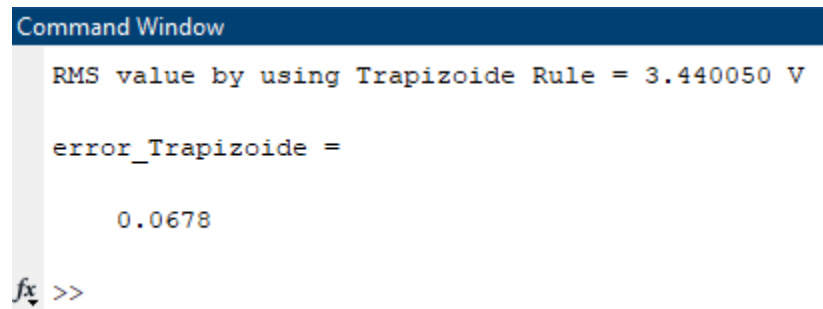
a = h/2*( v(1) + v(end) + 2*sum(v(2:end-1)) );
```

```
fprintf("RMS value by using Trapizoide Rule = %f V\n", a);
```

```
exact_value = 3.6902;
```

```
error_Trapizoide = ((exact_value-a)./exact_value)
```

OUTPUT:



```
Command Window  
RMS value by using Trapizoide Rule = 3.440050 V  
  
error_Trapizoide =  
  
0.0678  
fx >>
```

Figure 2.1: RMS Value and error shown using simpson $\frac{1}{3}$ rule.

(ii)

Code:

```
clear all;
```

```
close all;
```

```
clc;
```

```
t = [0, 0.125, 0.25, 0.375, 0.5, 0.6250, 0.7500, 0.8750, 1];
```

```
v = [0, 1.5949, 2.6960, 3.4404, 3.9390, 4.2764, 4.5110, 4.6752, 4.7750];
```

```
xi = t(1);    %% lower limit
```

```
xf = t(end);  %% upper limit
```

```
h = 0.125;    %% step size
```

```
v2 = v.^2;    %% square of voltage values
```

```
a = h/3*(v2(1) + v2(end) + 4*sum(v2(2:2:end-1)) + 2*sum(v2(3:2:end-2)));
```



```

Vrms = sqrt(a/(xf - xi));

fprintf("RMS value by using Simpson 1/3rd Rule = %f V\n", Vrms);

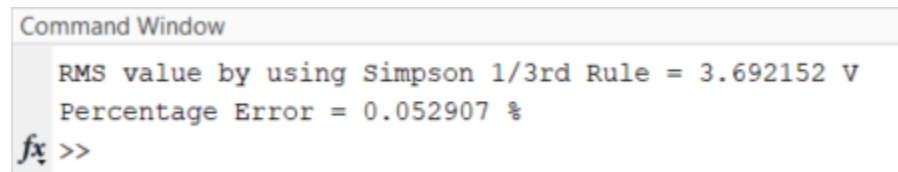
exact_value = 3.6902;

error = abs((exact_value - Vrms)./exact_value) * 100;

fprintf("Percentage Error = %f %%\n", error);

```

OUTPUT:



```

Command Window

RMS value by using Simpson 1/3rd Rule = 3.692152 V
Percentage Error = 0.052907 %
fx >>

```

Figure 2.2: RMS value and Error is shown.

As we get less error percentage using Simpson $\frac{1}{3}$ rule, I would choose Simpson $\frac{1}{3}$ rule over trapezoidal rule to ensure better accuracy.

Discussion:

In this question, the RMS value of the voltage signal was calculated in two different ways to compare accuracy. First the RMS value was determined using the discrete voltage values from Table 2, where Simpson's $\frac{1}{3}$ rule was chosen because it works well with equally spaced data and provides better accuracy than the trapezoidal rule. The obtained RMS value was very close to the true value of 3.6908 V, resulting in a small error. Next the RMS value was calculated using the voltage time equation obtained from interpolation, where numerical integration again showed improved accuracy due to the smooth nature of the function. In both cases, Simpson's $\frac{1}{3}$ rule produced less error proving it is a more accurate numerical approach for RMS calculation of smoothly varying signals.

Objective: The aim of this experiment is to learn the applicability of different numerical approaches and to calculate the value of current for a given value of capacitance by numerically differentiating the reconstructed voltage signal. Also to compare the real given value of current with the approximated value by using interpolation and plotting.

Question 3

At the reconstruction process, the voltage is applied across a R-C filter. To determine the current through the capacitor, following equation is considered:

$$i_C(t) = C \frac{dv_C(t)}{dt}$$

- i. Explain the applicability of different Numerical Differentiation approaches to determine the current from the completed data table (Table – 2) obtained from Question 1.
- ii. Considering the value of capacitor $C = 10 \mu\text{F}$, determine the current for each of the time instants.

If the true values of the current are as follows,

Table - 3

$i_C(\text{mA})$	155	105.207	71.409	48.469	32.898	22.330	15.156	10.287	6.983
$t(\text{ms})$	0	0.125	0.25	0.375	0.5	0.625	0.75	0.875	1

plot the true current (Table – 3) and your approximated current obtained from Question 3(ii) in the same figure window to make a comparison.

Answer:

(i)

Different numerical differentiation techniques are applied to estimate the derivative from discrete voltage data partially when analytical differentiation is not easily possible. These methods approximate the derivative at different points based on available data. They are -

1. Forward Difference Method

The Forward Difference method uses the next data points to approximate the derivative. For a small time step h , the forward difference formula is:

$$\frac{dv(t)}{dt} \approx \frac{v(t+h)-v(t)}{h}$$

Here $h = t_{i+1} - t_i$. This approach is most applicable at the beginning of the dataset, where backward data points are not available. Although it's simple, it is less accurate than other methods as it is biased forward and only first-order accurate.

2. Central Difference Method

The Central Difference Method estimates the derivative using data points on both sides of 't', providing a more balanced and accurate estimate:

$$\frac{dv(t)}{dt} \approx \frac{v(t+h)-v(t-h)}{2h}$$

Here, $2h = t_{i+1} - t_{i-1}$. This method is second-order accurate and used for those points where both previous and next values are available. It is the preferred method when applicable, due to its lower truncation error and symmetric structure.

3. Backward Difference Method

Backward Difference Method is used at the end of the dataset, where a future point is not available, the derivative is approximated using the previous point:

$$\frac{dv(t)}{dt} \approx \frac{v(t)-v(t-h)}{h}$$

Here, $h = t_i - t_{i-1}$. This approach is suitable for the last data point. It is first-order accurate like the forward difference and slightly less precise than the central difference method.

(ii)

Code:

Main Script:

```
clear all;
close all;
clc;
v = [0 2.696 3.939 4.511 4.775];%Sampling array
t = [0 0.25 0.5 0.75 1];%Sampling array
Time_s = 0:0.125:1; %Sampling array with step size 0.125.
Num = length (Time_s);
Voltage = volt_f_t (v,t,Time_s,Num);%Calling Function for interpolation.
%Showing the result.
fprintf("Time(ms)\t");
fprintf("%.4f\t\t",Time_s);
fprintf("\n");
fprintf("Volt(Volts)\t");
fprintf("%.4f\t\t",Voltage);
fprintf("\n");
current_ic = zeros(1,Num);%Allocating array to store value.
fprintf("Calculated value of current using the formula ic = C* d(v)/dt\n");
C = 10e-6;
T=Time_s.*10^(-3);%Converting the time to second from mili second.
for r = 1: Num
if r==1%Forward Difference Method.
Rate_voltage(r)=(Voltage(r+1)-Voltage(r))/(T(r+1)-T(r));
elseif r>=2 && r<=(Num-1)%Central Difference Method.
Rate_voltage(r)= (Voltage(r+1)- Voltage(r-1))/(T(r+1) - T(r-1));
elseif r==Num %Backward Difference Method.
Rate_voltage(r) = (Voltage(r)-Voltage(r-1))/(T(r)-T(r-1));
end
current_ic(r) = C* Rate_voltage(r);%Calculating values of approximated Current
in Ampere.
current_ic_mA(r) = current_ic(r)*10^(3);%Current expressed in mA from A.
fprintf("For time =%f ms, approximated current ic = %f
mA\n",Time_s(r),current_ic_mA(r));
end
%Current(approx) vs time_s
T_continous = 0:0.0000001:10^(-3);%Time in second.
NS_T = length(T_continous);
Aproxi_current = volt_f_t(current_ic,T,T_continous,NS_T);%Calling the function
for interpolation.
figure(2)
plot(Time_s,current_ic_mA,'w.','MarkerSize',20);%Points plotted in white color.
hold on;%To keep it intact in the graph.
Aproxi_current_mA= Aproxi_current*10^(3);%Current expressed in mA.
T_continuous_ms = T_continous*10^(3);%Time expressed in millisecond.
plot(T_continuous_ms,Aproxi_current_mA,'r.');
```

```

hold on;
%Current (real) vs time_s
True_current = 10^(-3).* [155 105.207 71.409 48.469 32.898 22.330 15.156 10.287
6.983];
True_current_interp = volt_f_t(True_current,T,T_continuous,NS_T);
True_current_interp_mA = True_current_interp*10^(3);
figure(2)
plot(T_continuous_ms,True_current_interp_mA,'b-');%Plotting real values in
graph.
hold on;
title('Current Ic Vs Time Graph');%Giving title of the graph.
xlabel('Time(ms)');%labeling x axis.
ylabel('Current Ic (mA)');%Labeling y axis.
legend('Plotted points','Approximated Current','True Current(Table 3)');

```

Function:

```

function vol = volt_f_t(V,T,T_s,N)
n = length (T);
vol = zeros (1,N);%Allocating array to store values.
%Lagrange interpolation formula
for k=1:N
P = ones(1,n);
for i = 1:n
for j = 1:n
if j~=i
P(i) = P(i) * (T_s(k)-T(j))/ (T(i) - T(j));
end
end
P(i) = P(i) * V(i);
end
vol(k) = sum(P);
end

```

Output:

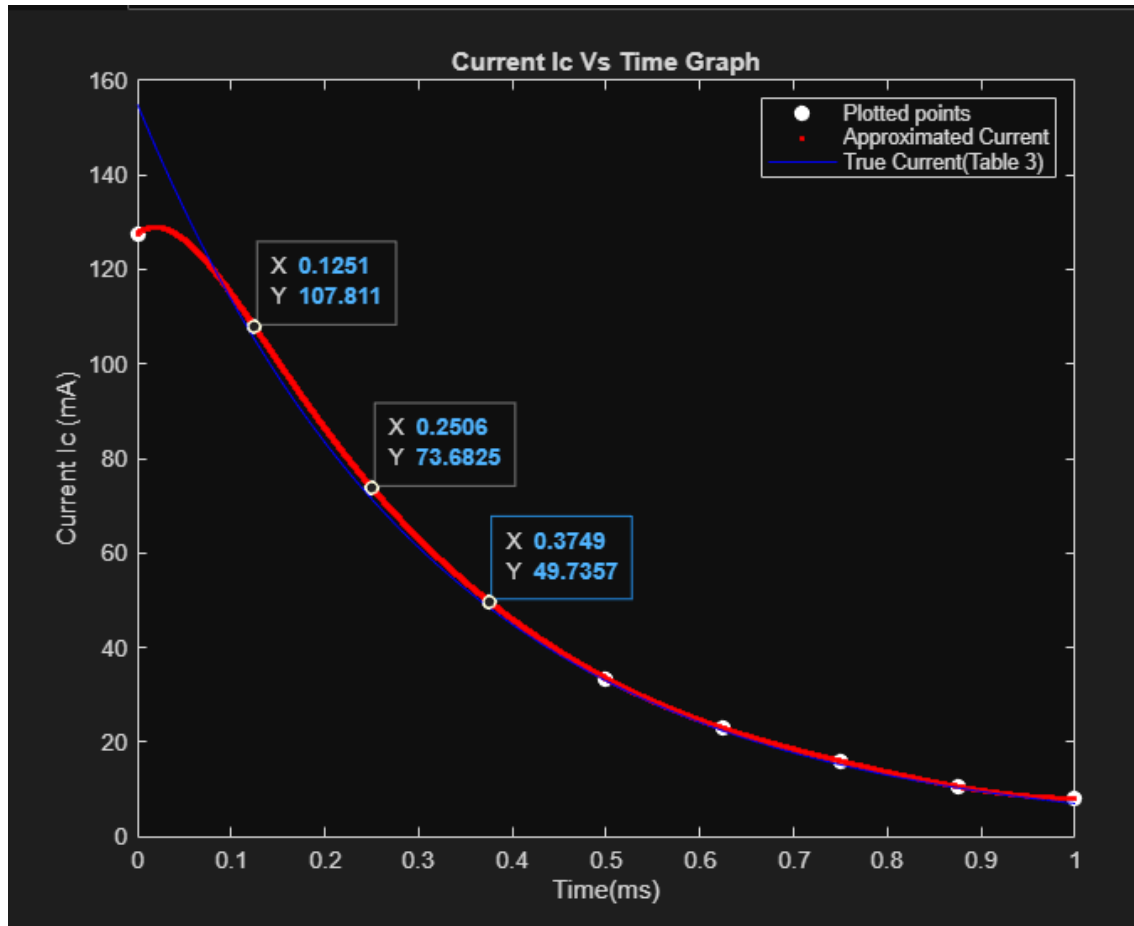


Figure 3.1: Plot showing the comparison between real and approximate values of current.

```
Command Window

New to MATLAB? See resources for Getting Started.

Calculated value of current using the formula  $i_c = C \cdot d(v)/dt$ 
For time =0.000000 ms, approximated current  $i_c$  = 127.589375 mA
For time =0.125000 ms, approximated current  $i_c$  = 107.840000 mA
For time =0.250000 ms, approximated current  $i_c$  = 73.822500 mA
For time =0.375000 ms, approximated current  $i_c$  = 49.720000 mA
For time =0.500000 ms, approximated current  $i_c$  = 33.437500 mA
For time =0.625000 ms, approximated current  $i_c$  = 22.880000 mA
For time =0.750000 ms, approximated current  $i_c$  = 15.952500 mA
For time =0.875000 ms, approximated current  $i_c$  = 10.560000 mA
For time =1.000000 ms, approximated current  $i_c$  = 7.985625 mA
>>
```

Figure 3.2: Approximate values of current at different times.

Discussion:

The voltage signal was numerically differentiated to determine the current through the capacitor in this experiment. Analytical differentiation was not possible for this experiment due to lack of discrete data points. Hence numerical differentiation - Forward Difference Method(for the first point, Central Difference Method(for the point in between) and Backward Difference Method(for the last point) was used. The central difference method is the most efficient one as it gives the least truncation error.

The interpolated voltage values obtained from Question 1 was used to get the value of current at each instant using the formula $i_c = C \times \frac{dv}{dt}$ where $C = 10$ micro Farad. From the plot we realise that the approximated current values closely follows the trend of the real values of current given in Table 3 showing the efficiency of interpolation.

Small deviations are expected as lower order forward and backward differentiation approximation is applied. To conclude, the experiment shows that the numerical approaches are quite effective in estimating derivatives and in analyzing circuit behavior when discrete values are given.

Objective: The objective of this experiment is to determine the time when the voltage across the device goes beyond 4 V by using a suitable numerical root-finding method(Bisection method) based on the interpolated voltage-time relationship.

Question 4

After running the voice analyzing device for a certain duration of time, the student observed that the device started to malfunction. Exploring the datasheet, s/he found that the optimum operating voltage for the device is 3.3V and the maximum voltage that it can sustain for long duration is 4.0V. With the help of the equation obtained from Question 1(ii), determine the time instant at which the voltage across the device crosses 4.0V, using a suitable Numerical approach.

CODE:

Function:

```
%% Function: Bisection method for nonlinear root
function ROOT = Sol_of_Non_Linear(f, x_lo, x_up, tol)
    format long

    % Check if bracket is valid
    if (polyval(f, x_lo) * polyval(f, x_up) >= 0)
        disp('Provide different bracket');
        ROOT = NaN;
        return;
    end

    % Initialization
    iter = 0;
    Iteration = [];
    XL = [];
    XU = [];
    XR = [];
    Error = [];
    result_new = 0;
    error = 100;

    % Bisection loop
    while (error > tol)
        iter = iter + 1;
        xr = (x_lo + x_up)/2;
```



```

Iteration = [Iteration; iter];
XL = [XL; x_lo];
XU = [XU; x_up];
XR = [XR; xr];

result_old = result_new;
result_new = xr;

if (polyval(f, xr) * polyval(f, x_lo) > 0)
    x_lo = xr;
elseif (polyval(f, xr) * polyval(f, x_lo) < 0)
    x_up = xr;
else
    break;
end

if iter > 1
    error = abs((result_new - result_old)/result_new) * 100;
else
    error = 100;
end

Error = [Error; error];
end

% Output
ROOT = xr;

% Display table
T = table(Iteration, XL, XU, XR, Error);
disp(T);
end

```

Main script:

```

clc;
clear all;
close all;

```

```

format short;

% Given data
t_given = [0 0.25 0.5 0.75 1];
v_given = [0 2.696 3.939 4.511 4.775];

% Polynomial interpolation
p = polyfit(t_given, v_given, length(t_given) - 1); % 4th-order polynomial

% Adjust polynomial: we want to find when voltage = 4
% So subtract 4 from the constant term
p(end) = p(end) - 4;

% Interpolation evaluation
interval = t_given(2) - t_given(1);
t = t_given(1):interval/2:t_given(end);
v = polyval(p + [0 0 0 0 4], t); % Add back 4 to plot original voltage

% Plot
figure(1);
plot(t, v, '+-k');
xlabel('Time (s)');
ylabel('Voltage (V)');
title('Interpolated Voltage vs Time');

% Root finding (i.e., time at which voltage = 4)
x_lo = 0.3;
x_up = 0.6;
tol = 1e-3;

voltage_4_at_t = Sol_of_Non_Linear(p, x_lo, x_up, tol);
fprintf('Voltage reaches 4V at t = %.4f seconds\n', voltage_4_at_t);

```

OUTPUT:

Iteration	XL	XU	XR	Error
1	0.3	0.6	0.45	100
2	0.45	0.6	0.525	14.2857142857143
3	0.45	0.525	0.4875	7.69230769230769
4	0.4875	0.525	0.50625	3.70370370370369
5	0.50625	0.525	0.515625	1.81818181818182
6	0.515625	0.525	0.5203125	0.900900900900914
7	0.515625	0.5203125	0.51796875	0.452488687782823
8	0.51796875	0.5203125	0.519140625	0.225733634311521
9	0.519140625	0.5203125	0.5197265625	0.112739571589632
10	0.519140625	0.5197265625	0.51943359375	0.0564015792442317
11	0.519140625	0.51943359375	0.519287109375	0.0282087447108561
12	0.519287109375	0.51943359375	0.5193603515625	0.0141023833027653
13	0.519287109375	0.5193603515625	0.51932373046875	0.00705168887946954
14	0.51932373046875	0.5193603515625	0.519342041015625	0.00352572012832766
15	0.519342041015625	0.5193603515625	0.519351196289062	0.0017628289879556
16	0.519351196289062	0.5193603515625	0.519355773925781	0.000881406725120487

Voltage reaches 4V at t = 0.5194 seconds

Figure 4.1: Output showing different values of XL,XU, XR and Error.

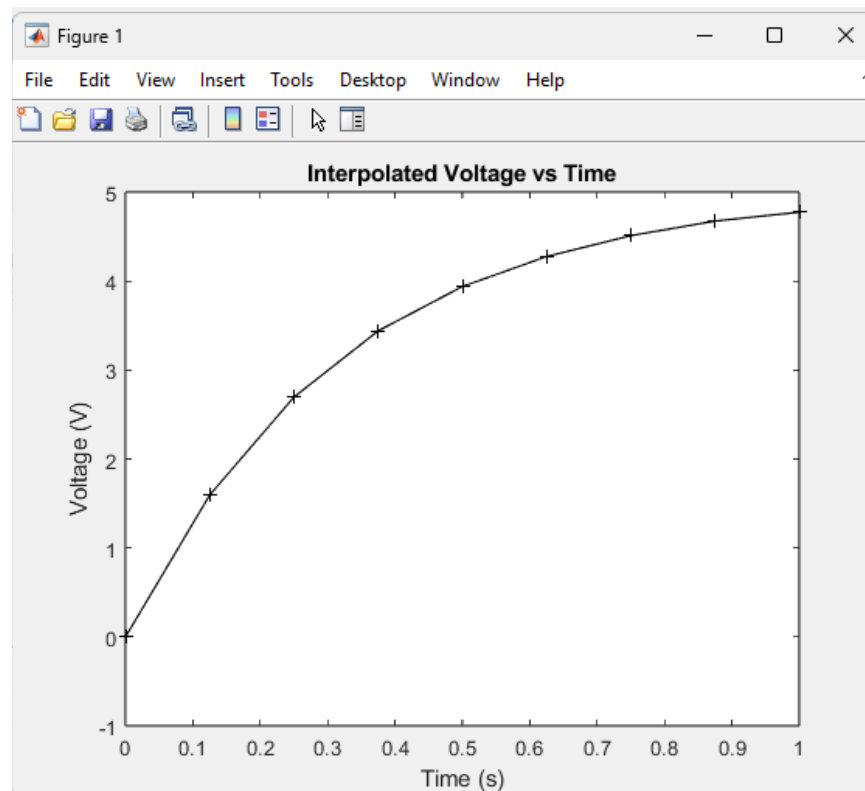


Figure 4.2: Plot showing Voltage vs Time graph.

Discussion:

In this question, a numerical root-finding method was used to determine the time when the voltage across the device exceeds 4.0 V which is the maximum safe operating limit. Since the voltage time relationship obtained from interpolation is continuous. The bisection method was selected because it is simple, stable, and guarantees convergence when a valid interval is chosen. By selecting a time range where the voltage changes from below 4.0 V to above 4.0 V, the method repeatedly reduces the interval until the required accuracy is achieved. The result clearly identifies the exact time at which the device voltage crosses the safe limit, helping to explain the cause of malfunction and showing the usefulness of the bisection method in practical life problems.